

Tutorial_6_Multi_File_Atomic_Edits

Tutorial 6: Multi-File Atomic Edits

Duration: 20 minutes **Level:** Intermediate

Overview

Perform atomic changes across multiple files with automatic rollback: - Transaction-based editing
- Cross-file refactoring - Automatic rollback on failure - Conflict detection

Step 1: Start Transaction

```
helixcode edit transaction start
```

```
# Transaction ID: tx-20251106-001
```

Step 2: Add Multiple Edits

```
# Rename function across all files
```

```
helixcode edit add --transaction tx-20251106-001 \  
  --files "internal/**/*.go" \  
  --search "ProcessPayment" \  
  --replace "HandlePayment"
```

```
# Update import paths
```

```
helixcode edit add --transaction tx-20251106-001 \  
  --files "**/*.go" \  
  --search "github.com/old/payment" \  
  --replace "github.com/new/payment"
```

```
# Update tests
```

```
helixcode edit add --transaction tx-20251106-001 \  
  --files "**/*_test.go" \  
  --search "TestProcessPayment" \  
  --replace "TestHandlePayment"
```

Step 3: Preview Changes

```
helixcode edit preview tx-20251106-001
```

```
# Files to be modified: 23
```

```
# Total changes: 47
```

```
#
```

```
# internal/payment/service.go: 5 changes
```

```
# internal/order/handler.go: 3 changes
# tests/payment_test.go: 2 changes
# ...
```

Step 4: Commit or Rollback

```
# Run tests first
make test

# If tests pass, commit
helixcode edit commit tx-20251106-001

# ✓ Transaction committed
# ✓ 23 files modified
# ✓ 47 changes applied

# If tests fail, rollback
helixcode edit rollback tx-20251106-001

# ✓ Transaction rolled back
# ✓ All files restored
```

Step 5: AI-Powered Multi-File Refactoring

```
helixcode refactor --multi-file --transaction \
  --task "Extract email sending logic into a separate
        EmailService
        - Create internal/email/service.go
        - Update all callsites
        - Add configuration
        - Write tests"

# HelixCode analyzes codebase
# Creates transaction
# Edits all affected files atomically
```

Step 6: Conflict Detection

```
# If Git conflict detected
# HelixCode aborts transaction

# Error: Conflict detected in internal/payment/service.go
# Line 45: Both HEAD and transaction modified
# Transaction aborted: tx-20251106-002
```

Advanced: Conditional Edits

```
// Use HelixCode API for complex logic
tx, _ := multiedit.NewTransaction()

// Add edit only if condition met
files, _ := fs.ReadDir("internal/services")
for _, file := range files {
    content, _ := os.ReadFile(file.Name())
    if strings.Contains(string(content), "logger.Error") {
        tx.AddEdit(&multiedit.FileEdit{
            Path: file.Name(),
            Search: "logger.Error",
            Replace: "logger.ErrorWithContext",
        })
    }
}

tx.Commit()
```

Results

- **Safety:** Atomic commits or full rollback
- **Consistency:** All files updated together
- **Speed:** Bulk changes in seconds

Continue to [Tutorial 7: Distributed Development](#)

Sources verified

Sources verified 2026-05-29: <https://go.dev/dl/> (go1.26.3 latest stable Go) ; project go.mod (inner go 1.26) + CLAUDE.md §3.1. **NEGATIVE FINDING** — this tutorial documents HelixCode's own atomic multi-file edit workflow and references no external third-party service or versioned dependency; nothing stale to fix. Reviewed against project version authority only.